



Adaptive Traffic Light Controller

Design & Implementation Using Verilog HDL on FPGA

SUBJECT: VLSI Design | CODE: BTEC13404 | SEM: IV | 2025–2026

ET24BTEC003 – Siya Bhalala

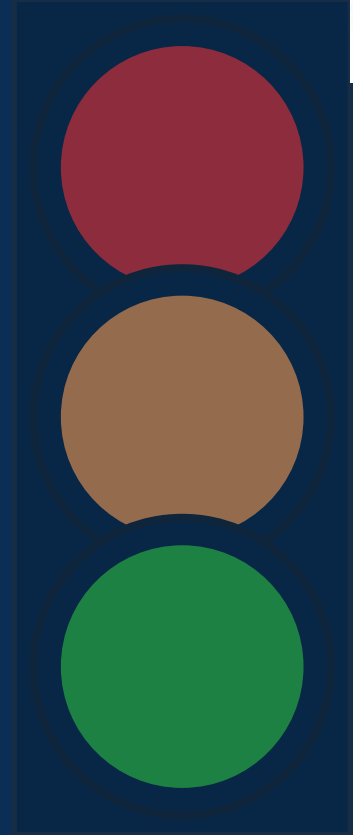
ET24BTEC011 – Khushi Desai

ET24BTEC019 – Haiya Patel

ET24BTEC034 – Jharna Nakrani

Prof.(DR.) Nehal Shah | Prof. Chintan Panchal | Prof. (DR.) Ketki Pathak

Branch: Electronics & Communication



Problem Statement

Design Adaptive traffic light controller for a 4-way junction. Draw a state diagram, state table and write Verilog code to demonstrate results on hardware using FPGA. Make your own PCB for demonstrating the scenario appropriately.



Control System

Design a finite state machine to manage signal sequencing at a 4-way road junction safely.



HDL Implementation

Implement the design in Verilog HDL and verify functionality using simulation waveforms.



Hardware Prototype

Deploy on FPGA and demonstrate with a custom-designed PCB that physically models the junction.

Why is This Important?



Urban traffic congestion costs billions in lost productivity annually



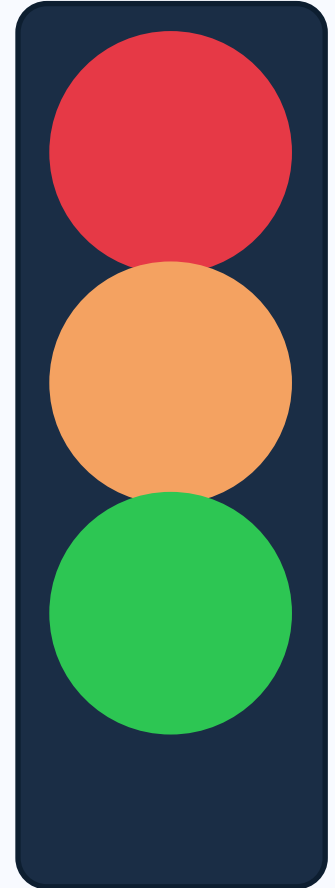
Manual signal systems are error-prone and inefficient



Automated FSM-based controllers ensure fairness and reduce accidents



FPGA implementation provides real-time, reliable operation



Real-World Applications



Smart City Traffic Management

Centralized FSM-based control for optimizing signal flow across entire urban networks.



Highway On-Ramp Metering

Automated controllers that regulate vehicle entry onto highways to prevent congestion.



Emergency Vehicle Preemption

Dynamic signal override to clear intersections for ambulances, fire engines, and police.



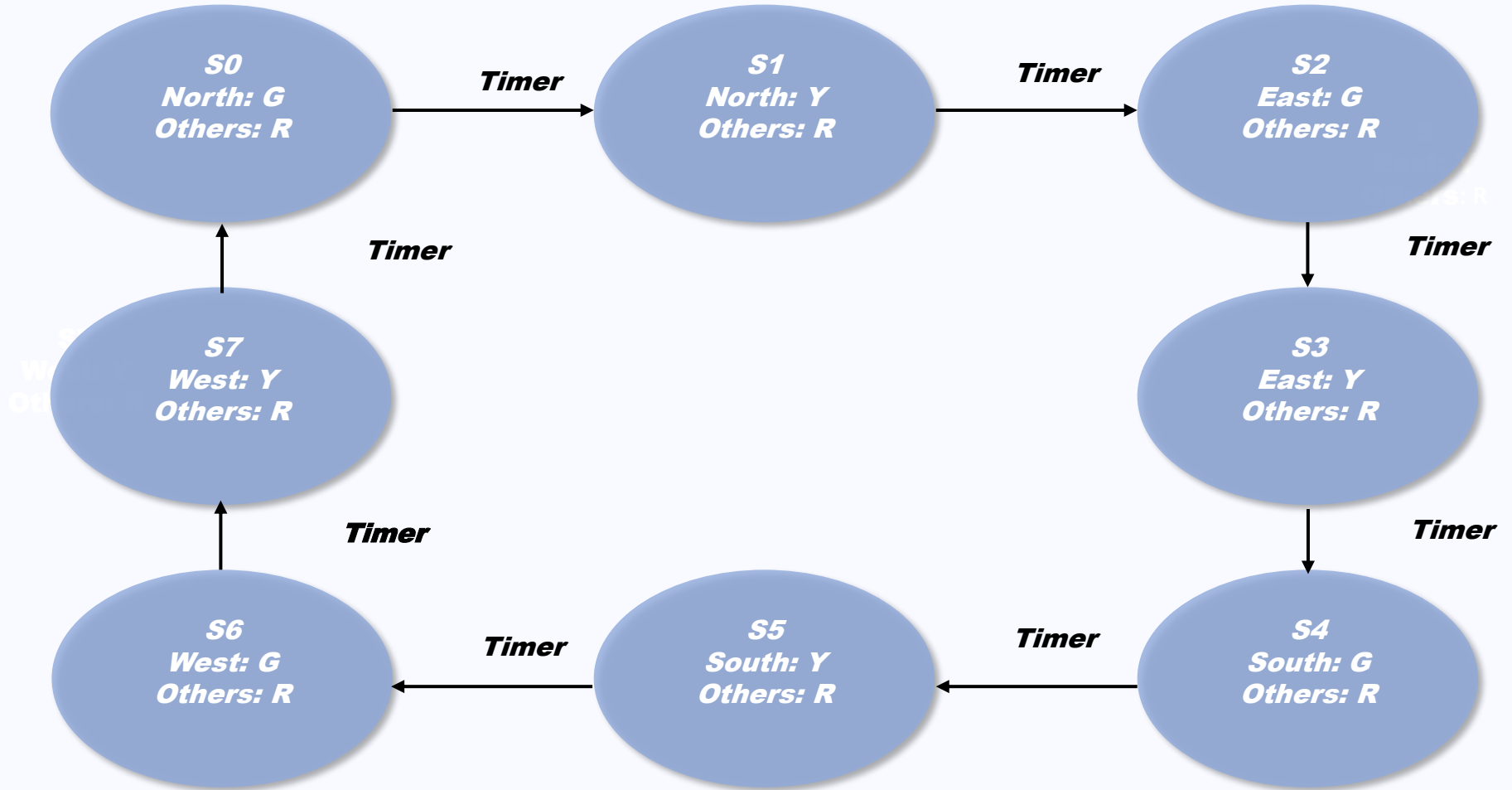
Pedestrian-Aware Adaptive Control

Sensor-driven signal adjustment that accounts for pedestrian crossing requests and flow.

State Table

State	NORTH	EAST	SOUTH	WEST	Next
S0	GREEN	RED	RED	RED	S1
S1	YELLOW	RED	RED	RED	S2
S2	RED	GREEN	RED	RED	S3
S3	RED	YELLOW	RED	RED	S4
S4	RED	RED	GREEN	RED	S5
S5	RED	RED	YELLOW	RED	S6
S6	RED	RED	RED	GREEN	S7
S7	RED	RED	RED	YELLOW	S0

State Diagram



Strategy in Implementation



Overall System Architecture

- State Register – stores the current state
- Timer / Counter – controls phase duration
- Output Decoder – generates R/Y/G signals for each road



FSM Design

- Moore FSM: outputs depend only on current state, not inputs
- 8 states: NG, NY, EG, EY, SG, SY, WG, WY



Timing Control

- Counter driven by FPGA clock increments each cycle
- State transitions when counter reaches max (GREEN_TIME or YELLOW_TIME)

Verilog Code – Sequential Block

```
module
Traffic_Light
(input clk, input reset,
 output reg [2:0] north, east, south, west );

    parameter
RED=3'b100, YELLOW=3'b010, GREEN=3'b001;
    parameter
NG=0, NY=1, EG=2, EY=3, SG=4, SY=5, WG=6, WY=7;
    reg [2:0] state; reg [31:0] counter;

    always
@(posedge clk or posedge reset) begin
    if(reset) begin state <= NG; counter <= 0; end
    else begin counter <= counter + 1;
        case(state)
            NG: if(counter >= GREEN_TIME) begin state<=NY; counter<=0; end
            NY: if(counter >= YELLOW_TIME) begin state<=EG; counter<=0; end
            // ... EG, EY, SG, SY, WG, WY follow same pattern
        endcase end end
```


Verilog Code – Combinational Block

```
always @(*) begin
    north = RED;  east = RED;
    south = RED;  west = RED;

    case(state)
        NG: north = GREEN;
        NY: north = YELLOW;
        EG: east  = GREEN;
        EY: east  = YELLOW;
        SG: south = GREEN;
        SY: south = YELLOW;
        WG: west  = GREEN;
        WY: west  = YELLOW;
    endcase
end
endmodule
```

How It Works

1 Sequential Block

Updates state & counter on every clock edge. Resets on posedge reset.

2 Combinational Block

Pure output logic — sets all signals to RED then overrides the active direction.

3 Two always Blocks

Clean separation of timing (sequential) and output (combinational) logic.

Verilog Code – Complete Module (with Density Inputs)

```
`timescale 1ns / 1ps
module Traffic_Light(
    input clk, input reset,
    input north_density, input east_density,
    input south_density, input west_density,
    output reg [2:0] north, east, south, west);

    parameter RED=3'b100, YELLOW=3'b010, GREEN=3'b001;
    parameter NG=0, NY=1, EG=2, EY=3, SG=4, SY=5, WG=6, WY=7;
    reg [2:0] state; reg [31:0] counter;
    parameter GREEN_CHECK=200_000_000, YELLOW_TIME=200_000_000;
    // Active LOW sensors
    wire vN=~north_density, vE=~east_density;
    wire vS=~south_density, vW=~west_density;

    always @(posedge clk or posedge reset) begin
        if(reset) begin state <= NG; counter <= 0; end
        else begin counter <= counter+1;
            case(state)
                NG: begin if(counter>=GREEN_CHECK) begin counter<=0;
                    if(!vN) state<=NY; else state<=NG; end end
                NY: if(counter>=YELLOW_TIME) begin state<=EG; counter<=0; end
                EG: begin if(counter>=GREEN_CHECK) begin counter<=0;
                    if(!vE) state<=EY; else state<=EG; end end
                EY: if(counter>=YELLOW_TIME) begin state<=SG; counter<=0; end
                SG: begin if(counter>=GREEN_CHECK) begin counter<=0;
                    if(!vS) state<=SY; else state<=SG; end end
                SY: if(counter>=YELLOW_TIME) begin state<=WG; counter<=0; end
                WG: begin if(counter>=GREEN_CHECK) begin counter<=0;
                    if(!vW) state<=WY; else state<=WG; end end
                WY: if(counter>=YELLOW_TIME) begin state<=NG; counter<=0; end
            endcase end end

    always @(*) begin
        north=RED; east=RED; south=RED; west=RED;
        case(state)
            NG:north=GREEN; NY:north=YELLOW; EG: east=GREEN; EY: east=YELLOW;
            SG:south=GREEN; SY:south=YELLOW; WG: west=GREEN; WY: west=YELLOW;
```

Key Features

Density Inputs

Active LOW sensor per direction. vN/vE/vS/vW are inverted inputs.

Adaptive Timing

Stays GREEN if traffic detected; advances if no vehicles.

Clock: 100 MHz

GREEN_CHECK = 2s (200M cycles). YELLOW = 2s.

Outputs: 3-bit

RED=100, YELLOW=010, GREEN=001 per direction.

8 FSM States

NG NY EG EY SG SY WG WY (N→E→S→W cycle).

XDC Constraints File – Complete Pin Mapping

```
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVC MOS33 [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVC MOS33 [get_ports reset]

# North LEDs
set_property PACKAGE_PIN J1 [get_ports {north[0]}]
set_property PACKAGE_PIN L2 [get_ports {north[1]}]
set_property PACKAGE_PIN J2 [get_ports {north[2]}]

# East LEDs
set_property PACKAGE_PIN G2 [get_ports {east[0]}]
set_property PACKAGE_PIN H1 [get_ports {east[1]}]
set_property PACKAGE_PIN K2 [get_ports {east[2]}]

# South LEDs
set_property PACKAGE_PIN H2 [get_ports {south[0]}]
set_property PACKAGE_PIN G3 [get_ports {south[1]}]
set_property PACKAGE_PIN A14 [get_ports {south[2]}]

# West LEDs
set_property PACKAGE_PIN A16 [get_ports {west[0]}]
set_property PACKAGE_PIN B15 [get_ports {west[1]}]
set_property PACKAGE_PIN B16 [get_ports {west[2]}]

# Density Sensors
set_property PACKAGE_PIN K3 [get_ports north_density]
set_property PACKAGE_PIN M3 [get_ports east_density]
set_property PACKAGE_PIN M1 [get_ports south_density]
set_property PACKAGE_PIN N1 [get_ports west_density]

set_property IOSTANDARD LVC MOS33 [get_ports *]
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

Pin Map Summary

Signal	Pin	Std
clk	W5	LVC33
reset	U18	LVC33
north[0..2]	J1, L2, J2	LVC33
east[0..2]	G2, H1, K2	LVC33
south[0..2]	H2, G3, A14	LVC33
west[0..2]	A16, B15, B16	LVC33

Density Sensors

north_density	K3	LVC33
east_density	M3	LVC33
south_density	M1	LVC33
west_density	N1	LVC33

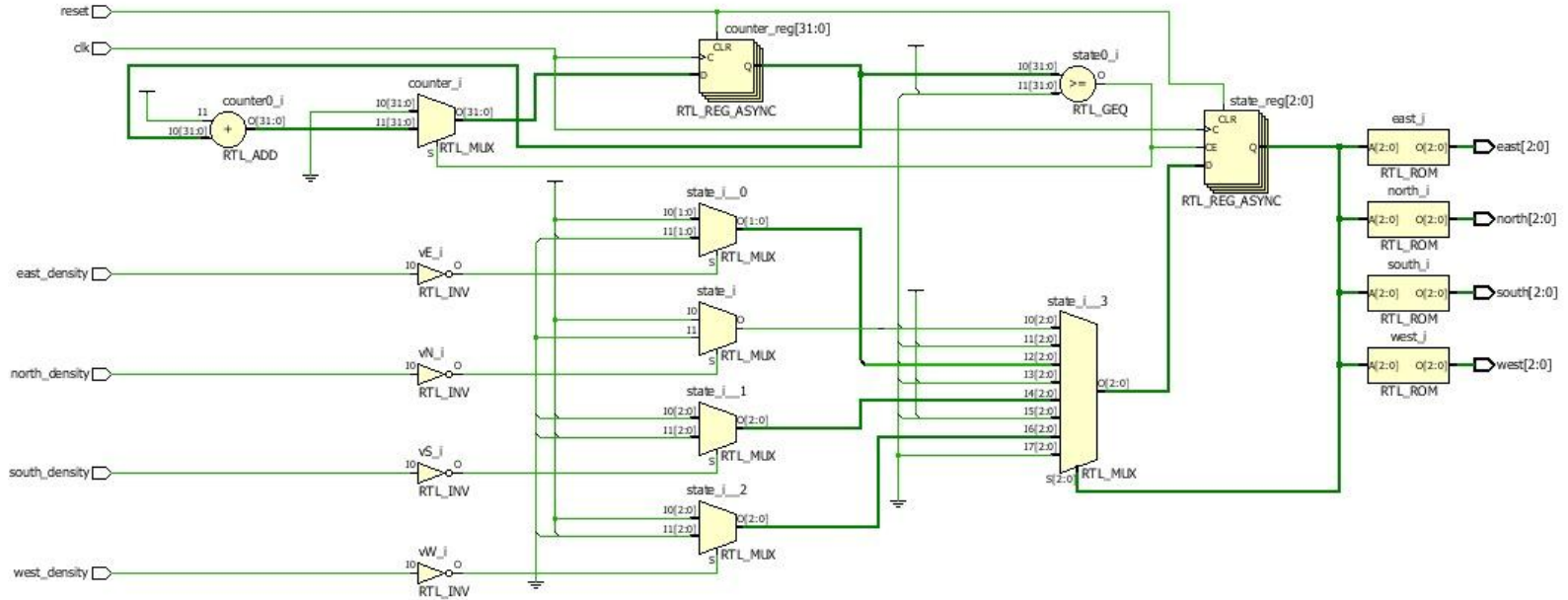
Config

CFGBVS=VCCO, CONFIG_VOLTAGE=3.3V. All ports LVC MOS33.

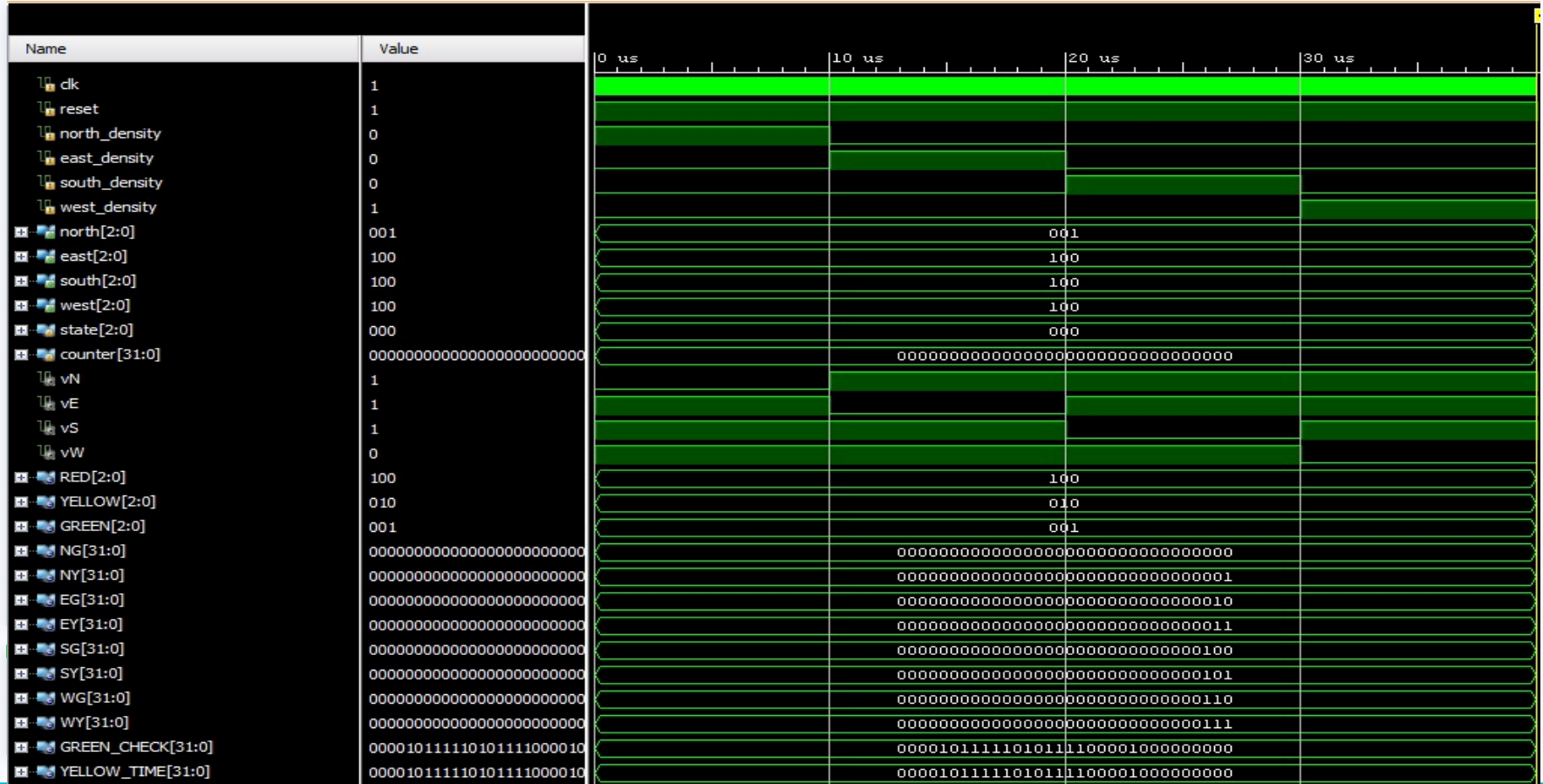
Target Board

Basys 3 (Artix-7 XC7A35T). 3.3V LVC MOS I/O standard throughout.

RTL Diagram



Output Waveform



TCL Script

```
# Run 1: Normal operation
restart
add_force {/Traffic_Light/clk} \
    -radix bin {1 0ns} {0 5000ps} \
    -repeat_every 10000ps
add_force {/Traffic_Light/reset} \
    -radix bin {0 0ns}
run 10 us

# Run 2: Apply reset
add_force {/Traffic_Light/clk} \
    -radix bin {1 0ns} {0 5000ps} \
    -repeat_every 10000ps
add_force {/Traffic_Light/reset} \
    -radix bin {1 0ns}
run 10 us
```

Script Purpose

- Drives the clock at 100 MHz (10 ns period, 5 ns half)
- First run simulates normal operation for 10 μ s
- Second run tests asynchronous reset behavior
- Used in Vivado Behavioral Simulation console

XDC Constraints File

```
set_property PACKAGE_PIN W5  [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property PACKAGE_PIN U18 [get_ports reset]
set_property IOSTANDARD LVCMOS33 [get_ports reset]

# North
set_property PACKAGE_PIN J1 [get_ports {north[0]}]
set_property PACKAGE_PIN L2 [get_ports {north[1]}]
set_property PACKAGE_PIN J2 [get_ports {north[2]}]

# East
set_property PACKAGE_PIN G2 [get_ports {east[0]}]
set_property PACKAGE_PIN H1 [get_ports {east[1]}]
set_property PACKAGE_PIN K2 [get_ports {east[2]}]

# South / West (similar pattern...)
set_property IOSTANDARD LVCMOS33 \
    [get_ports {north[*] east[*] south[*] west[*]}]

set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]
```

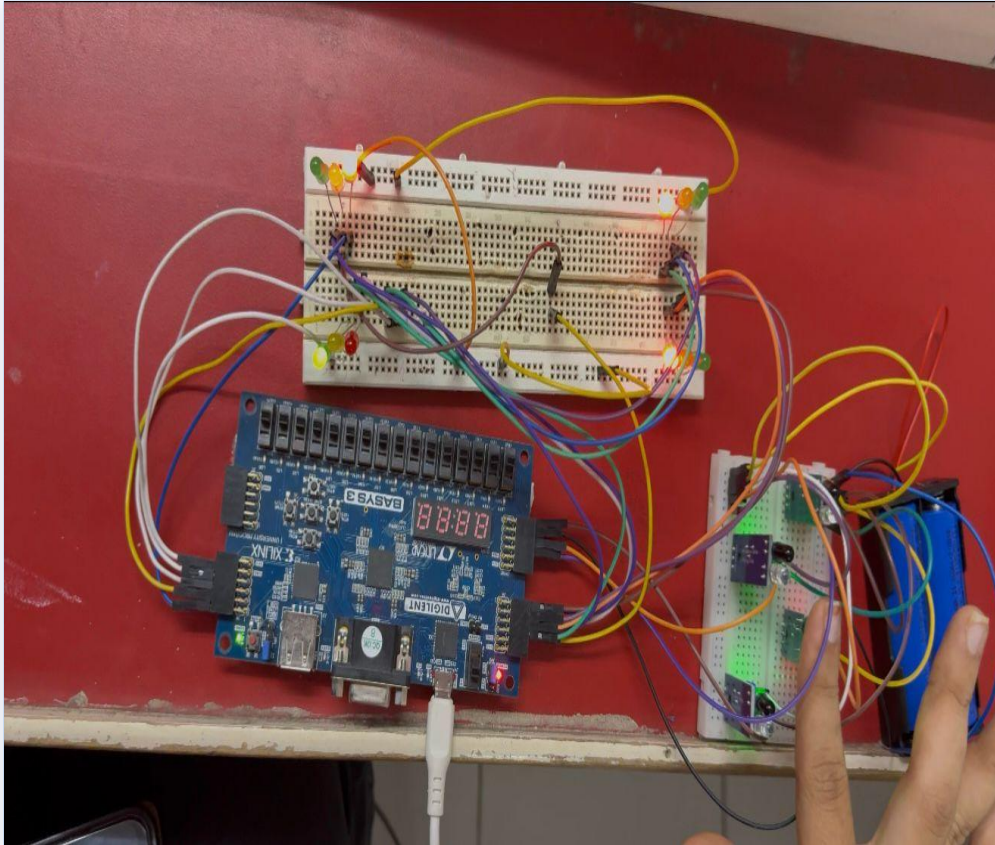
Pin Mapping Summary

Signal	Pin	I/O Std
clk	W5	LVCMOS33
reset	U18	LVCMOS33
north[0..2]	J1,L2,J2	LVCMOS33
east[0..2]	G2,H1,K2	LVCMOS33
south[0..2]	H2,G3,A14	LVCMOS33
west[0..2]	A16,B15,B16	LVCMOS33

Comparison with Existing Designs

Parameter	Manual Control	Timer-Based IC	This Design (FPGA)
Control Method	Human operator	555-timer ICs	FSM Verilog / FPGA
Flexibility	High (manual)	Fixed timing	Fully reprogrammable
Reliability	Low	Medium	High
Cost	High (labor)	Low	Medium
Scalability	Poor	Poor	Excellent
Emergency Override	Manual only	Not possible	Easily programmable
Hardware	Traffic booth	Discrete ICs	Single FPGA + PCB
Simulation	Not possible	Limited	Full waveform sim

Hardware Implementation



1

Create Vivado Project

New project targeting Artix-7 (Basys 3)

2

Add Verilog Source

Add Traffic_Light.v and constraints .xdc

3

Synthesis & Implementation

Run synthesis, place-and-route, check timing

4

Generate Bitstream

Generate .bit file from implemented design

5

Program FPGA via USB

Load bitstream; SW0=reset, SW1=run

6

Observe on PCB / LEDs

Verify R/Y/G signals cycle on all 4 directions

System Architecture & Final Working

State Register

Stores current FSM state (3-bit). Clocked sequential register updated on posedge clock.

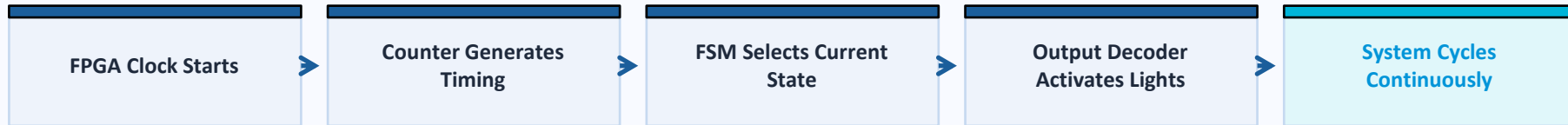
Timer / Counter

32-bit counter increments every clock cycle. Resets and triggers state change at GREEN_TIME or YELLOW_TIME.

Output Decoder

Combinational logic: sets all outputs to RED by default, then overrides the active direction.

Final Working Sequence



PCB Demonstration:

A custom PCB was designed to represent the 4-way junction. LEDs on the PCB represent traffic lights controlled by FPGA GPIO outputs, with current-limiting resistors for each signal pin.

Concluding Remarks



Successfully designed and implemented a 4-way traffic light controller using a Moore FSM.



Verified correct functionality through simulation and waveform analysis in Xilinx Vivado.



Achieved reliable real-time operation using FPGA-based hardware implementation on Basys 3.



Demonstrated practical applicability through a custom PCB model of a traffic junction.



Ensured safe and efficient traffic flow with no signal conflicts and proper timing control.



Future: adaptive timing, sensor-based optimization, pedestrian phases, larger FPGA platforms.

Thank You!

Adaptive Traffic Light Controller | VLSI Design

Prof.(DR.) Nehal Shah | Prof. Chintan Panchal | Prof. (DR.) Ketki Pathak

